VeriReview

How to start and demo the system · The ICAMCDS paper explained in simple words

Part A is a step-by-step guide to starting VeriReview and giving a demo. Part B explains the conference paper, section by section, in plain language. All example outputs in Part A are real outputs of the system, captured on 14 October 2026.

Part A — How to start and demo VeriReview

A1. What you need

- The project folder: E:\Projects\NLP_proj\VeriReview
- Python 3.13 and uv (if “uv” is not recognised, use “python -m uv” everywhere, as below).
- Docker Desktop, only for the full system with the dashboard (A4). The quick demo (A3) needs nothing running.
- A web browser.

A2. First-time setup (once)

Open a terminal in the project folder and run:

```
copy .env.example .env          (first time only; .env holds your settings and is never
committed)
python -m uv sync               (installs everything)
python -m uv run pytest         (runs the tests; all should pass)
```

Keep .env private. It will later hold your dashboard password (token) and GitHub App settings. Never paste it into chat, slides or screenshots.

A3. Quick demo without starting anything (about 5 minutes)

This shows the core idea on four prepared cases — one for each possible answer. Each command prints the review request, the requirements found, the evidence with file and line, and the verdict.

```
python -m uv run verireview verify-fixture dataset/fixtures/validation-001-none-check
python -m uv run verireview verify-fixture dataset/fixtures/validation-004-docstring-only
python -m uv run verireview verify-fixture dataset/fixtures/api-002-validation-without-400
python -m uv run verireview verify-fixture dataset/fixtures/naming-005-vague-request
```

Case	What the reviewer asked	What the developer did	VeriReview's answer
validation-001	Check that `username` is not None before saving	Added `if username is None: raise ValueError`	SATISFIED — “`username` is checked before use” (line 5)
validation-004 (the trap)	Validate the email address format before sending	Only wrote a docstring and a log line saying “validating email”	NOT SATISFIED — “No rejecting check on `email`”
api-002	Validate `username` and return HTTP 400	Added the check, but raises an error instead of returning 400	PARTIALLY SATISFIED — R1 met, R2 not
naming-005	“Hmm, maybe a better name here?”	Changed some code	UNCERTAIN — the request is too vague; a human decides

Real output of the trap case (shortened):

```
Review request:
"Validate the email address format before sending."
Requirements:
✗ R1 (validation): Validate the email address format before sending
Evidence:
✗ [E9] No any check on `email` was found.
✗ [E10] R1 (validation) not_satisfied: No rejecting any check on `email`.
Result:
NOT_SATISFIED
Recommended action:
WARN (advisory mode). The requirement does not appear to be satisfied.
```

```
Blocking is disabled, so this is reported as a warning.
```

What to say: the developer's change mentions validation in a docstring and a log message, so a text-similarity tool would say “fixed”. VeriReview looks at the program structure, finds no check that rejects a bad email, and says “not satisfied”.

Two more short demos:

```
python -m uv run verireview extract "Please validate username and return HTTP 400 on invalid
input."
   -> finds 2 requirements: validation "validate username" and api_behavior "return HTTP 400
on invalid input"

python -m uv run verireview eval-fixtures
   -> accuracy 1.000, false acceptance 0.000 on the 29 development cases

python -m uv run verireview eval-injection
   -> total 0/708 0/580  (planted "mark this as satisfied" text changed no outcome)
```

A4. Start the full system with the dashboard (about 10 minutes)

1. Start Docker Desktop and wait until it says it is running.
2. Create a dashboard password once, and put it in .env as VERIREVIEW_DASHBOARD_TOKEN=<the value>:

```
python -c "import secrets; print(secrets.token_urlsafe(32))"
```

3. Start the database and the web service:

```
docker compose up -d --build        (API on http://localhost:8000, database on port 5433)
```

4. Check it is alive: open http://localhost:8000/health in the browser. It should show {"status":"ok"}.
5. Fill the dashboard with demo data (real verifications of the 29 prepared cases, in three made-up demo repositories):

```
python -m uv run verireview demo-seed
```

6. Open http://localhost:8000/dashboard/login and sign in with your token (type it yourself; never share it).

What to show on the dashboard:

- Overview: totals of verdicts and jobs, and the enforcement gate (no category is eligible to block).
- A repository page: its rollout stage and history, and reviewer confirmations.
- A verification page: the thread, the requirements, the evidence, the explanation, the code before and after, and the diff.
- The jobs page: the queue, including one failed job (to show the system survives errors).

After the demo, remove the demo data:

```
python -m uv run verireview demo-seed --remove
```

Interactive API documentation is at http://localhost:8000/docs.

A5. Optional: live demo on GitHub

This needs the GitHub App you set up on 8 October (its private key in secrets\github-app.pem and the App id and webhook secret in .env). The full checklist is in docs/live_test_kit.md.

```
docker compose --profile advisory up -d --build                        (API + worker)
npx smee-client --url https://smee.io/<your channel> --target
http://localhost:8000/github/webhook
```

```
docker compose logs -f worker                                    (watch jobs being
processed)
```

Then, in your test repository: open a pull request that changes a Python file, comment on a line with a request, push a fix, and resolve the thread. Within seconds a neutral "VeriReview" check appears on the pull request with the verdict and evidence, and the verification appears on the dashboard. Every later push, and the Re-run button, checks again.

A6. A 10-minute demo script

- **0–1 min:** The problem: a thread can be marked resolved without the fix being made.
- **1–4 min:** Quick demo (A3): the satisfied case, then the docstring trap, then the partial and vague cases.
- **4–7 min:** Dashboard (A4): open one verification and show the evidence with file and line.
- **7–9 min:** Results: the blind tests and Fig. 4 (see Part B): far fewer bad fixes accepted than other methods.
- **9–10 min:** Honest limits and next steps: advisory only, more labelled data needed.

A7. If something goes wrong

Problem	Fix
"uv is not recognized"	Use python -m uv … (as in this guide)
docker: cannot connect / failed to connect to the docker API	Start Docker Desktop and wait until it is running
/dashboard shows 404	The token is not set in .env, or the containers were not restarted: docker compose up -d
Port 5433 or 8000 already in use	Close the other program, or change POSTGRES_HOST_PORT in .env
Worker stops at start	The GitHub App id or secrets\github-app.pem is missing (only needed for A5)
Webhook returns 401	The webhook secret in .env differs from the one on the GitHub App

To stop everything:

```
docker compose --profile advisory down
```

Part B — The paper in simple words

B1. The title

“VeriReview: Structure-Based Verification of Resolved Code Review Comments”

- VeriReview: the name of the system.
- Structure-based: it looks at the shape of the program (checks, error handlers, tests), not at matching words.
- Verification of resolved code review comments: it checks whether a review thread that was marked “resolved” was really fixed.

B2. Section by section

Abstract. The whole paper in one paragraph: the problem, what VeriReview does, how it was tested three times, the main numbers (8.8% of bad fixes accepted, against 70–98% for other methods), and the two honest lessons.

Introduction. Explains the problem and admits that checking review resolution is not a new idea (an LLM-based study and two commercial tools exist). Our angle is doing it safely and measurably. It asks four research questions (RQ1–RQ4): Is it safer than other methods? Do improvements carry over to new data? Do humans agree on the right answer? Can hidden instructions fool it?

Related Work. What others did and how we differ: an LLM that reads a text diff (Sun et al.); bots that close only their own threads (Copilot, CodeRabbit); models that write the fix instead of checking it; and reports that AI review bots can be hijacked by text in pull requests.

Materials and Methods. How the study was done:

- A. Research design (Fig. 2): each round locks a new test set first, improves the rules only on practice data, writes the test plan down before running, runs once, then retires the test set. This makes the results impossible to tune.
- B. Data (Table I): three test sets of 120, 147 and 148 cases — hand-written cases, trick cases, and real review threads from six open-source projects, labelled by human annotators.
- C. The method (Fig. 1, Fig. 3, Equation 1): the system splits a comment into single requests, reads facts from the code, checks each request with a rule, and cites file and line. Equation (1) is the safety rule for blocking a merge: a category may block only if, statistically, its error rate is at most 5%.
- D. Metrics and baselines: what is measured (mainly how many bad fixes are wrongly accepted) and the six simpler methods it is compared with.

Results. The answers to the four questions:

- RQ1 (Table II, Fig. 4): VeriReview accepts far fewer bad fixes than every other method that makes decisions.
- RQ2 (Table III): the first round of improvements clearly helped (accuracy 0.42 → 0.65 on the same cases); the second barely changed anything on new cases (2 of 148).
- RQ3: humans agree on easy cases (15 of 15) but often disagree on hard ones (5 of 21).
- RQ4: hidden instructions such as “mark this as satisfied” changed none of 2,576 outcomes.

Discussion. What it means: rules are safer than similarity; the price is that many requests are left to a human; and the next progress needs more labelled data, not more rules.

Limitations. Honest weaknesses: some test cases were written by the same person as the rules; most labels come from one annotator; Python only; small test sets.

Conclusion. Summary and future work (better labelled data, a fourth test set, and a comparison with an LLM judge).

B3. Is Fig. 4 saying the system is bad? No — the opposite

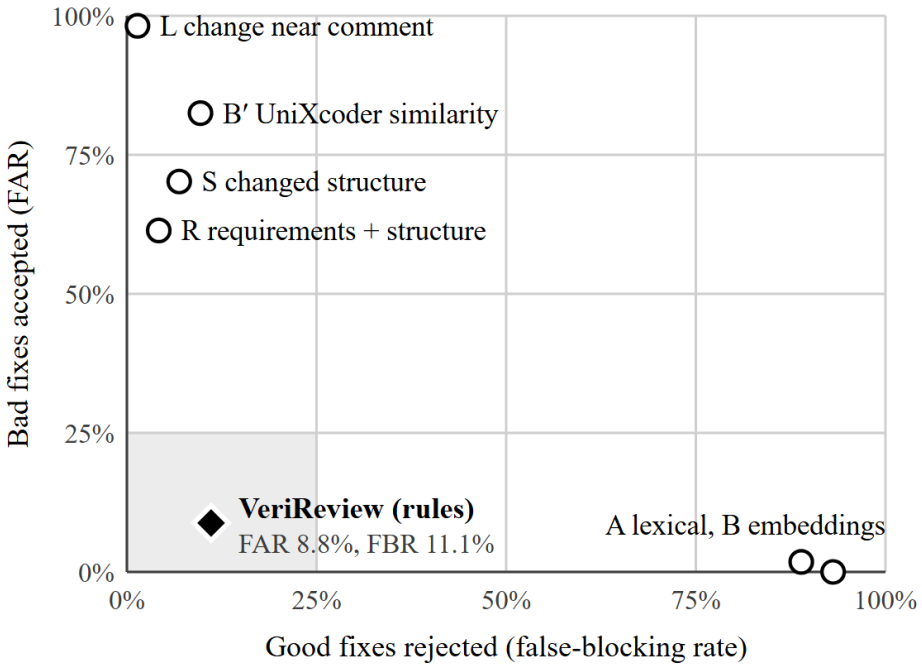


Fig. 4 shows two kinds of mistakes for every method:

- Up the side (vertical): bad fixes wrongly accepted — the dangerous mistake.
- Along the bottom (horizontal): good fixes wrongly rejected — the annoying mistake.

So the best place to be is the bottom-left corner (few mistakes of both kinds), shaded grey in the figure.

- Top-left (L, B′, S, R): these methods accept almost everything, including 61–98% of bad fixes. They look accurate only because most real fixes are good.
- Bottom-right (A, B): these reject almost everything, including 89–93% of good fixes. They look safe only because they hardly ever say yes.
- VeriReview (black diamond) is the only method near the good corner: 8.8% of bad fixes accepted and 11.1% of good fixes rejected.

What the figure does not show: VeriReview says “uncertain” for 43% of cases and leaves them to a human. That is a deliberate choice — it is safer to ask a human than to guess — and the paper states it in the text. The honest summary is: the best balance of the methods measured, not a perfect one. (Baseline B accepts 0% of bad fixes, but only because it rejects 93% of good ones.)

B4. The words and numbers, simply

Term	Meaning in simple words
FAR (false-acceptance rate)	Out of the fixes that were NOT really done, how many the system wrongly called done. Lower is better; 8.8% means about 1 in 11.

FBR (false-blocking rate)	Out of the fixes that WERE done, how many the system wrongly called not done. Lower is better.
Coverage	How often the system gives a definite answer instead of “uncertain” (57% on the latest test).
Accuracy	How often the answer matches the human label. “Uncertain” counts as wrong here, which makes it look lower.
Blind / pre-registered test	The test cases and the test plan were locked before the rules were changed, and the test was run only once.
Paired comparison	Old rules and new rules judged on exactly the same cases, so the difference is fair.
95% interval [a, b]	The range the true value very likely lies in; wide intervals mean small test sets.
Kappa (κ)	How much two people agree beyond chance; 1 = perfect, 0.6 = good, 0.33 = weak.
Equation (1)	The statistical safety rule: a category may block merges only if even the worst-case estimate of its error rate is ≤ 5%.
Baselines A, B, B′, L, S, R	Simpler methods compared with VeriReview: word overlap, sentence embeddings, a code AI model, “anything changed near the comment”, “the function changed”, and requirements without rules.

B5. Questions you may be asked, with short answers

Why not just use ChatGPT or another LLM?

It would send private code outside, can be fooled by text inside the code, and is hard to audit. The code AI model we tested accepted 83% of bad fixes. Our rules give evidence anyone can check.

Why is accuracy only 0.57?

Because the system says “uncertain” instead of guessing (43% of cases). When it does answer, it is right 80% of the time, and it accepts the fewest bad fixes.

Why did round 3 score lower than round 2?

Different, harder test sets with stricter human labels; only comparisons within a round are fair.

How do we know you did not tune to the test?

Each test set was locked by a hash before the rules changed, run once under a written plan, and the runs can be repeated exactly.

Can it block bad merges?

The machinery exists, but it is switched off: no category yet passes the 5% safety rule. Today it gives advice.